

LRR Full System Test Plan

Date: 2026-09-10 · **Last updated:** 2026-09-13 (added §10.5 — retry refused once a sibling attempt already succeeded — and §16, the new Audit Log) **Scope:** Every customer/operator/staff-facing flow currently implemented, on staging. Manual, WhatsApp + admin dashboard driven — the automated Jest suite (356 unit + 152 integration tests, `lrr-service`) already covers unit-level logic and real-database behaviour; this plan verifies the real, end-to-end experience a real user would have, including the one thing the automated suite cannot: that money actually moves at Paystack, not just in our own database.

How to use this: Work top to bottom. Each numbered case has Setup → Steps → Expected Result. Check the box when the actual result matches. If it doesn't, stop, note what actually happened, and file it rather than continuing past a failure in that flow — later cases often assume the earlier ones worked.

Do not reset between scenarios. Run the whole plan as one continuous session with the same test operator and motorist numbers. A run that starts clean each time will pass while real users fail: the bugs found on 2026-09-12 were all cases where state from an earlier job leaked into a later one, and none of them are reachable on a first pass. §13 exists specifically to test that accumulated history, so leave it until last.

You'll need:

- A test motorist WhatsApp number (not staff, not an existing operator)
- A test operator WhatsApp number, registered and `ACTIVE` in the admin Operators tab
- Admin dashboard access (ADMIN or SUPER_ADMIN role)
- `disputeAlertPhoneNumber` set in Platform Settings, and access to that number's WhatsApp — it now receives dispute alerts, low-rating alerts, *and* stalled-confirmation alerts
- A real (or test-mode) Paystack card for payment steps
- Access to the Paystack dashboard for the same mode (test/live) the environment under test actually uses — several steps below only pass if the money movement shows up there, not just in our own UI
- A SUPER_ADMIN bearer token for one direct API call in §14 (there's no dashboard button for it yet, and this endpoint moves platform money out so it's SUPER_ADMIN-only, same as payouts) — grab it from your browser's dev tools (Storage) while logged in as a SUPER_ADMIN, or however your team normally makes authenticated calls against the API

1. Request Creation & Media

- ☐ **1.1 — Start a request** Send `SOS` from the test motorist number. **Expected:** Bot asks for a location pin.
- ☐ **1.2 — Share location** Share a live or dropped pin. **Expected:** Bot asks for issue type (breakdown/accident/flat tyre/fuel).
- ☐ **1.3 — Pick issue type, vehicle type, destination** Follow prompts through to destination. **Expected:** Bot asks for photo/video evidence.
- ☐ **1.4 — Upload media, then type `DONE`** Send 1-2 photos, then `DONE`. **Expected:** Bot proceeds to deposit/dispatch (exact next step depends on subscriber status — see §2).
- ☐ **1.5 — Missing location, text-only landmark** Restart a fresh request; when asked for location, send a text landmark instead of a pin. **Expected:** Bot explicitly asks for a pin drop rather than silently accepting text as the location.

2. Deposit Payment

- ☐ **2.1 — Initial deposit message wording** Reach the deposit-payment step. **Expected:** Message says "You have 5 minutes" (not 30) before the request is cancelled.
- ☐ **2.2 — First reminder (real 5-minute mark)** Wait 5 real minutes without paying. **Expected:** A reminder arrives saying "You have 5 more minutes to complete payment".
- ☐ **2.3 — Second reminder (15-minute mark)** Continue waiting to the 15-minute mark. **Expected:** A reminder arrives with no specific number of minutes mentioned.
- ☐ **2.4 — Final warning (25-minute mark)** Continue to the 25-minute mark. **Expected:** Reminder arrives with "Your request will be cancelled soon", still no exact number.
- ☐ **2.5 — Hard cutoff (30-minute mark)** Continue to 30 minutes without paying. **Expected:** Request is cancelled; motorist told payment wasn't received in time; operator told the job is no longer available.
- ☐ **2.6 — Successful payment** On a fresh request, pay the deposit within the window. **Expected:** Motorist gets operator confirmation; operator gets "payment confirmed, job is live." **Then open the Paystack dashboard's Transactions list and confirm a successful charge exists for this exact amount, dated just now** — our own confirmation message says what we think happened, not what actually happened at Paystack.
- ☐ **2.7 — Re-prompt if motorist messages before paying** On a fresh request, send any random text before paying (don't use the payment link). **Expected:** Bot re-sends the "please pay" prompt, mentioning 5 minutes.

3. Dispatch & Bidding

- ☐ **3.1 — Operator receives offer** With a fresh request reaching dispatch, confirm the test operator receives a dispatch-offer message with vehicle type, destination, distance, and media link.
- ☐ **3.2 — Operator quotes** Reply with a bare number (e.g. `50000`). **Expected:** Motorist receives the quote in their shortlist.
- ☐ **3.3 — Operator declines** On a separate request/offer, reply `no` or `decline`. **Expected:** Operator is told the decline was recorded; motorist's shortlist doesn't include this operator.

4. Operator Selection & Assignment

- ☐ **4.1 — Motorist selects a quote** Reply with the shortlist number of a quote. **Expected:** "Operator selected!" message, deposit link, "You have 5 minutes" (same wording check as §2.1, this is the actual trigger point).

5. Masked Chat Relay (CHAT DRIVER) — new this pass

- ☐ **5.1 — Start a chat before deposit is paid** Right after selecting a quote (before paying deposit), send `CHAT DRIVER` as the motorist. **Expected:** Both motorist and operator get "You're now connected... Reply `END CHAT` anytime to stop."
 - ☐ **5.2 — Messages relay correctly** As the motorist, send a plain message (e.g. "Are you close?"). **Expected:** The operator receives it as `Customer: Are you close?` — from LRR's number, not the motorist's real number.
 - ☐ **5.3 — Reverse direction** As the operator, reply with a plain message. **Expected:** The motorist receives it as `Driver: ...` — from LRR's number, not the operator's real number.
 - ☐ **5.4 — Commands are swallowed during relay, not executed** While still in the chat, have the operator send `arrived`. **Expected:** This is relayed as a chat message (`Driver: arrived`) to the motorist — it does **not** trigger the real `ARRIVED` flow. (This is deliberate — confirms the explicit-exit design is working, not a bug.)
 - ☐ **5.5 — END CHAT exits for both sides** Either party sends `END CHAT`. **Expected:** Both sides receive "Chat ended." Operator can now send `arrived` and have it work as a real command (see §6).
 - ☐ **5.6 — No operator assigned yet** On a brand new request before any operator is selected, send `CHAT DRIVER`. **Expected:** Bot replies that no operator is assigned yet — no relay starts.
-

6. Dispatch & En-Route

- ☐ **6.1 — Operator sends ARRIVED** With deposit paid and outside any active chat relay, operator sends `arrived`. **Expected:** Motorist is notified their operator has arrived.
 - ☐ **6.2 — ARRIVED rejected in wrong state** Have a different, unassigned operator try `arrived`. **Expected:** Told they don't have an active job.
-

7. Completion & Confirmation

- ☐ **7.1 — Operator sends DONE** Operator sends `done`. **Expected:** Motorist gets "job is done, reply `CONFIRM` or `DISPUTE`."
 - ☐ **7.2 — Motorist confirms** Reply `CONFIRM`. **Expected:** Status moves to Completed immediately (check admin dashboard); balance payment link sent.
 - ☐ **7.3 — Stalled confirmation → staff alert, NOT auto-complete** On a fresh `DONE`, don't reply for 30 real minutes. **Expected:** `disputeAlertPhoneNumber` receives an alert ("customer hasn't confirmed... please check on them"). Confirm in the admin dashboard the request status is **still Arrived**, not auto-completed, and the operator was **not** told to release the vehicle.
-

8. Dispute Intake & Resolution

- ☐ **8.1 — Raise a dispute** After a `DONE`, reply `DISPUTE` instead of `CONFIRM`. **Expected:** Status becomes `In Dispute` (check admin dashboard). Motorist told to expect contact, asked for their

side. Operator told **not to release the vehicle**, asked for their side (confirm this goes to the *operator*, not repeated to the motorist).

- ☐ **8.2 — Both statements captured** As motorist, send a free-text explanation. As operator, send a free-text explanation. **Expected:** Both appear on the request detail in the admin dashboard, correctly labeled "Customer said" / "Operator said."
- ☐ **8.3 — Staff resolve with an adjustment** In the admin dashboard, open the disputed request, enter a resolution note and a settlement percentage (e.g. 60). **Expected:** A payment link for the adjusted amount is sent to the motorist. Status is still `In Dispute` (not yet Completed). Operator gets a message confirming the settlement amount.
- ☐ **8.4 — Payment completes the disputed job** Motorist pays the settled amount. **Expected:** Status becomes `Completed`. Both original and settled balance amounts are visible on the request detail.
- ☐ **8.5 — Confirm during an unresolved dispute is blocked** On a fresh dispute (before staff resolve it), have the motorist reply `CONFIRM`. **Expected:** Told the request is still under dispute review — job does **not** complete, no payment link sent.
- ☐ **8.6 — Confirm after resolution, before payment** After staff resolve (8.3) but before paying, have the motorist reply `CONFIRM` again. **Expected:** Told to use the payment link already sent — does **not** trigger a second link or complete the job early.
- ☐ **8.7 — Reopen a resolved dispute** On an already-resolved-but-unpaid dispute, motorist replies `DISPUTE` again. **Expected:** Dispute reopens; staff re-alerted; motorist told it's reopened (not a fresh "logged" message).

9. Balance Payment & Vehicle Release

- ☐ **9.1 — Successful balance payment (undisputed job)** Complete a normal (non-disputed) job through to balance payment. **Expected:** Motorist gets payment confirmation + rating prompt. Operator gets "release the vehicle" + rating prompt. **Confirm in the Paystack dashboard's Transactions list** that a successful charge for the balance amount exists, separate from the deposit's transaction in §2.6.

10. Payout

- ☐ **10.1 — Successful payout** With an operator who has bank details on file, confirm a payout fires after balance payment. **Expected:** Operator receives "payment sent" with the amount; payout shows `Success` in the admin Payouts tab. **Then open the Paystack dashboard's Transfers list and confirm a successful transfer exists for this exact amount, to this operator's bank account** — check the recipient and amount match, not just that some transfer exists.
- ☐ **10.2 — Missing bank details** Repeat with an operator who has no bank details on file. **Expected:** Payout shows `Blocked` with reason "No bank details on file" in the admin Payouts tab; operator is notified to add bank details. (This used to show as `Pending` — if you see `Pending` here, that's a regression, not the expected state.)

- ☐ **10.3 — Retry after fixing bank details** Add bank details for the blocked operator, then click Retry in the admin Payouts tab. **Expected:** Payout proceeds to `Success`. Confirm in the Paystack dashboard as in 10.1.
 - ☐ **10.4 — Payout on a disputed request shows the dispute badge** Find a payout tied to a request that was disputed (from §8). **Expected:** The Payouts tab shows a "Disputed" or "Dispute Resolved" badge next to that job.
 - ☐ **10.5 — Retry is refused once a sibling attempt already succeeded** Find (or create) a job with more than one payout row — e.g. a `Failed` attempt followed by a `Success` one for the same job (10.3's flow naturally produces this: the original `Blocked` row and the row Retry created).
Expected: The **older, non-succeeded row never shows a Retry button** — instead it reads "Paid via a different attempt." This holds even if you filter the list down to just that older row's status.
Check the audit log (§16) for a `payout_retried` entry if you did trigger a retry attempt against it directly via the API — it should show the refusal, not a second transfer.
-

11. Rating & Low-Rating Flagging — new this pass

- ☐ **11.1 — Normal rating (3+ stars)** As motorist, rate the operator 4 or 5. **Expected:** Thank-you message with a feedback link. No staff alert.
 - ☐ **11.2 — Low rating from motorist triggers a flag + alert** On a different completed job, rate the operator 1 or 2. **Expected:** `disputeAlertPhoneNumber` receives an alert ("Customer rated the operator low"). In the admin dashboard, opening that request shows a red "Flagged" badge on the rating.
 - ☐ **11.3 — Low rating from operator triggers a flag + alert** As the operator, rate the motorist 1 or 2. **Expected:** Same alert pattern, message says "Operator rated the customer low." Flag visible in the dashboard.
 - ☐ **11.4 — Mark a flagged rating reviewed** In the admin dashboard, click "Mark Reviewed" on a flagged rating. **Expected:** Badge turns green ("Reviewed"), the button disappears, the flag record persists (doesn't just vanish).
-

12. Cross-Cutting Checks

- ☐ **12.1 — Admin dashboard dispute visibility (regression)** Confirm the Requests tab actually shows the "In Dispute" status and "Was Disputed" history correctly — this was previously silently broken (fields existed in the DB but weren't returned by the API) and was fixed this pass. Don't skip this check.
 - ☐ **12.2 — Duplicate SOS while a request is open** Send `SOS` again while an existing request is still active. **Expected:** Told about the existing active request rather than starting a second one.
 - ☐ **12.3 — Cancel an active request** Send `CANCEL` at various stages (before deposit, after deposit, mid-dispatch). **Expected:** Cancels cleanly each time; no charge if deposit wasn't paid; assigned operator (if any) notified.
-

13. Stale State Across Sequential Jobs

This whole section exists because of a real failure found on 2026-09-12. Running the plan top to bottom means the same test operator accumulates history — finished jobs, cancelled jobs, offers they never replied to. Several bugs only appear on the *second or third* run through a flow, never the first, so a clean single pass will not catch them.

Setup for this section: you need the test operator to have been through at least three earlier scenarios — one completed, one cancelled, and at least one where the operator was sent an offer and simply never replied to it. If you've worked through §1–§12 in order, you already have this.

- ☐ **13.1 — Open-job list contains only genuinely live jobs** Get a fresh offer out to the test operator, then reply with a bare price (e.g. 28000). **Expected:** Either the quote is accepted outright (if this is their only live offer), or the "you have N jobs open at once" list appears containing **only** jobs that are actually still live. A completed or cancelled job must never appear in that list. *This is the exact bug found on 2026-09-12: three old jobs were listed as open after they'd already ended.*
- ☐ **13.2 — Abandoned request releases the operator's offer** Start a request, let the operator receive the offer, and have the operator reply *nothing at all*. Let the request run until it auto-cancels (no operators left to try). **Expected:** Once the motorist is told the request was cancelled, that job no longer counts as open for the operator — verify by triggering 13.1 again and confirming it's absent from the list.
- ☐ **13.3 — Motorist-cancelled request releases the operator's offer** Same as 13.2, but the motorist sends CANCEL while the operator's offer is still unanswered. **Expected:** Same — the job disappears from the operator's open-job list immediately, not minutes later.
- ☐ **13.4 — Admin-cancelled request releases the operator's offer** Same again, but cancel from the admin dashboard (both the Cancel action and a manual status change to Cancelled) while the operator's offer is unanswered. **Expected:** Same as 13.3.
- ☐ **13.5 — Declining a new offer while a rating is still owed** Complete a job so the operator is prompted to rate the motorist, and **do not reply to that prompt**. Then send the operator a fresh dispatch offer for a different job and reply NO to decline it. **Expected:** The decline is recorded against the new job. The operator must **not** be told "please reply with a number from 1 to 5" — the outstanding rating prompt must not swallow the decline. *Found 2026-09-12: the stale rating prompt intercepted the decline and recorded a 1-star rating on the old job instead.*
- ☐ **13.6 — Quoting while a rating is still owed** Same setup as 13.5, but reply with a price (e.g. 30000) instead of NO . **Expected:** Treated as a quote on the new job, not as a star rating on the old one.
- ☐ **13.7 — Rating still works once no offer is live** With the rating prompt still outstanding and **no** live offers, reply with a number 1–5. **Expected:** Recorded as a rating as normal. (13.5/13.6 must not have broken the ordinary rating path.)
- ☐ **13.8 — Every operator-facing message names its job** Working through a job end to end, check each message the operator receives: dispatch offer, countdown notice, arrival confirmation, DONE/ARRIVED reminders, "payment received / release the vehicle", the rating prompt, and any "this job has already ended" reply. **Expected:** Every one of them names the job reference (e.g. Job #UH8DF7). An operator juggling several jobs must never get a message that just says "the job" or

"this customer" with no way to tell which. Motorist-facing messages don't need this — a motorist only ever has one active request.

14. Admin-Triggered Refund

There's no dashboard button for this yet — it's a direct API call only. Don't file "there's no Refund button" as a bug here; it's a known gap, tracked separately from this plan.

Setup: Pay a deposit (§2.6), then cancel the request from the admin dashboard (§12.3, the "after deposit" case). This leaves a request that's `Cancelled` with a successfully paid deposit — the exact "deposit arrived after the request was already cancelled" case this feature exists for.

- ☐ **14.1 — Trigger the refund** Using your `SUPER_ADMIN` bearer token, call `POST /rescue-requests/:id/refund-deposit` (no body) with the cancelled request's id from the setup above. An `ADMIN`-role token gets a 403 here — this is now `SUPER_ADMIN`-only, same as payouts. **Expected:** Returns `{"message": "Refund initiated"}`. **In the Paystack dashboard, find the original deposit's transaction and confirm a refund for the full amount now exists against it, eventually showing `Processed`.**
 - ☐ **14.2 — Refund cannot be double-triggered** Call the same endpoint again for the same request — once right after 14.1 (while the refund is still in flight), and once after 14.1 shows `Processed`. **Expected:** Both calls are rejected with a 400 (`"A refund is already in progress for this request"` while in flight, `"Not eligible for refund..."` once it's done). Neither call creates a second refund at Paystack — check the transaction's refund history, not just the API response.
 - ☐ **14.3 — Refunding a request with no paid deposit is rejected** Call the same endpoint on a cancelled request that never had its deposit paid. **Expected:** Rejected with `"Not eligible for refund – already refunded/in progress, or not a late-payment case."` — no refund attempt is made for money that was never collected.
-

15. Paystack Customer Identity

This isn't a user-visible flow — it's a data-integrity property that only the real Paystack dashboard can confirm. Paystack treats the email on a payment as the customer's identity, and that identity can never be changed later. A motorist who pays once as a guest and later registers a real email must stay the *same* Paystack customer — otherwise a card saved under the first identity can never be charged under the second, which only becomes visible once memberships/saved cards exist. The row-lock mechanics are already covered by `paystack-customer-identity.int-spec.ts`; this section checks the one thing only Paystack's own records can show.

- ☐ **15.1 — Guest pays, registers a real email, pays again — still one customer** As a fresh motorist with no email on file, complete a deposit payment (§2.6). Then register a portal account for that same phone number using a real email address. Then start a second request and pay its deposit too. **Expected:** In the **Paystack dashboard's Customers list**, search by phone number and by the real email just registered. Only **one** customer record exists for this person. Its email on file is the auto-generated `<digits>@lrr.ng` address from the *first* payment, not the real email registered afterward — that's expected, not a bug: the identity is frozen on first payment and Paystack has no way to update a customer's email later.
-

16. Audit Log

A durable, SUPER_ADMIN-only record of security-sensitive and financial admin actions — the kind that used to only page someone via Sentry. Each check below is "do X, then confirm an entry shows up for it," not a flow of its own.

- ☐ **16.1 — Staff creation is logged** Create a new staff account (`POST /auth/staff` , or wherever your admin UI exposes it). **Expected:** A new `Audit Log` entry appears with category `Staff Created` , the message names the role and email, and the actor is the SUPER_ADMIN who created it.
- ☐ **16.2 — Deposit refund is logged** Trigger §14.1 (admin-triggered refund). **Expected:** An entry appears with category `Deposit Refunded` , details include the rescue request id, actor is whoever called the endpoint.
- ☐ **16.3 — Payout retry is logged, including the outcome** Retry a payout (§10.3). **Expected:** An entry appears with category `Payout Retried` ; expand its details and confirm `resultStatus` matches what the Payouts tab actually shows afterward (e.g. `SUBMITTED` or `SUCCEEDED`), not just that a retry happened.
- ☐ **16.4 — Platform settings change is logged with the actual changed values** Change the service fee % or deposit % in Platform Settings. **Expected:** An entry appears with category `Platform Settings Updated` ; expanding details shows the new values you actually set, not a generic "settings changed" message.
- ☐ **16.5 — Dispute resolution is logged** Resolve a dispute (§8.3). **Expected:** An entry appears with category `Dispute Resolved` , details include the resolution note and settlement percentage you entered.
- ☐ **16.6 — Filtering by category narrows the list** Pick a category from the dropdown filter (e.g. "Payout Retried"). **Expected:** Only entries of that category show. Clearing the filter brings back everything.
- ☐ **16.7 — Marking an entry reviewed is purely bookkeeping** Click "Mark Reviewed" on any unreviewed entry. **Expected:** Its badge changes to "Reviewed" and the button disappears. Nothing else about the entry (category, message, actor, details) changes, and nothing elsewhere in the system is affected — this is a "someone looked at it" note, not an action.
- ☐ **16.8 — ADMIN cannot reach the audit log** Log in as an `ADMIN` (not `SUPER_ADMIN`) and try to open `/audit-log` directly by URL. **Expected:** Redirected away — the link isn't even in their nav menu, and the page itself refuses them.

Known Gaps — Not Testable Yet (features not built)

These will fail if tested, because they don't exist. Don't file these as bugs — they're tracked separately as open product decisions:

- `LOCATION ISSUE` keyword / masked voice calling
- Motorist no-show auto-cancel + operator fuel compensation after ARRIVED
- Off-platform settlement detection
- Blacklisting a customer who stalls on payment